pandas + NumPy Notes

`object` dtype

In pandas, `object` dtype means the column holds Python objects — in practice, almost always **strings**.

Pandas doesn't have a dedicated string dtype by default, so any column with text gets stored as `object`.

```
df.dtypes
# name      object   ← strings
# age        int64
# score    float64
```

You may also see `object` if a column has **mixed types** (e.g. some ints, some strings) — pandas falls back to `object` when it can't pick a single numeric type.

Boolean filtering — operator precedence

`&` and `|` bind tighter than comparison operators like `>`, `<`, `==`. Always wrap each condition in parentheses:

```
# Wrong — evaluates as: products['rating'] > (4.0 & products['in_stock'])
products[products['rating'] > 4.0 & products['in_stock']]

# Correct
products[(products['rating'] > 4.0) & (products['in_stock'])]
```

Counting nulls

```
df.isnull().sum()        # nulls per column (default)
df.isnull().sum(axis=1)  # nulls per row
```

NumPy functions vs array methods

Some operations are available both as NumPy functions and as array methods — they do the same thing:

```
arr.mean()       # method
np.mean(arr)     # function — same result
```

Basic stats available as both: `mean`, `sum`, `std`, `min`, `max`

More advanced operations are function-only (no array method):

```
np.median(arr)
np.percentile(arr, 75)
np.unique(arr)
np.where(condition, true_val, false_val)
```

NaN-safe NumPy functions

When working with plain NumPy arrays that contain NaN, use the `nan`-prefixed versions:

```
np.nanmean(arr)
np.nanmedian(arr)
np.nanstd(arr)
```

These ignore NaN values instead of returning nan.

Series vs plain array — important difference:

```
np.mean(df['col'])             # skips NaN — Series handles it
np.mean(df['col'].to_numpy())  # returns NaN — plain array doesn't
```

Converting to a plain array loses pandas' NaN-handling behavior.

`np.where` — compact if/else for arrays

Works like a vectorized if/else — operates on the entire array at once:

```
np.where(condition, value_if_true, value_if_false)

# Example
np.where(sensors['reading'] > 80, True, False)
```

Similar idea to lambda + apply, but faster because it's vectorized:

```
# lambda equivalent (slower, element by element)
sensors['reading'].apply(lambda x: True if x > 80 else False)
```

`pd.cut()` — right-inclusive bins

`pd.cut()` bins are **right-inclusive** (closed on the right) by default:

```
pd.cut(age, bins=[0, 17, 35, 60, 100], labels=['Minor', 'Young Adult',
'Adult', 'Senior'])
# bins are (0, 17], (17, 35], (35, 60], (60, 100]
# age=17 → 'Minor', age=35 → 'Young Adult'
```

The left edge is excluded, the right edge is included. Keep this in mind when setting bin boundaries.

Use `np.inf` as the upper bound when the last bin has no upper limit:

```
pd.cut(df['age'], bins=[0, 18, 35, 60, np.inf], labels=['Minor', 'Young',
'Adult', 'Senior'])
# last bin is (60, ∞] — captures everything above 60
```

`pd.pivot_table()` default aggfunc

`pd.pivot_table()` uses `aggfunc='mean'` by default — you don't need to specify it unless you want something else:

```
pd.pivot_table(df, values='tip_pct', index='sex', columns='day')
# mean (default)
pd.pivot_table(df, values='tip_pct', index='sex', columns='day',
aggfunc='sum')
pd.pivot_table(df, values='tip_pct', index='sex', columns='day',
aggfunc='count')
```

`groupby()` can produce the same values but returns a Series instead of a 2D table:

```
df.groupby(['sex', 'day'])['tip_pct'].mean()                        #
Series — same numbers, flat format
pd.pivot_table(df, values='tip_pct', index='sex', columns='day')   # 2D
table — easier to read
```

`pivot_table()` — NaN handling

NaN values in the data are ignored by default when computing the mean (equivalent to `skipna=True`). There's no `skipna` argument — to change this, pass a custom `aggfunc`:

```
tips.pivot_table(index='day', columns='time', values='tip_pct',
                 aggfunc=lambda x: x.mean(skipna=False))
```

`dropna` **parameter** — controls whether all-NaN *columns* are kept in the output, not whether NaNs in the data are skipped:

```
pivot_table(..., dropna=True)   # default — drops columns where every cell
is NaN
pivot_table(..., dropna=False)  # keeps those all-NaN columns
```

`fill_value` **parameter** — replaces NaN *result* cells after aggregation (doesn't affect the mean calculation):

```
pivot_table(..., fill_value=0)
```

`groupby().transform()` vs `groupby().apply()`

Both operate group-by-group, but they return different shapes:

	`apply()`	`transform()`
Output size	One row per group (reduced)	Same length as original DataFrame
Use case	Summarize each group	Add a per-group value back to each row

`apply()` **— reduces to one result per group:**

```
df.groupby('category')['price'].apply(np.mean)
# category
# A    15.0
# B    22.0
# dtype: float64
```

`transform()` **— broadcasts the group result back to every row:**

```
df['category_avg'] = df.groupby('category')['price'].transform('mean')
# Every row gets its group's mean — same length as df
```

`transform()` is the right tool when you want to add a group-level stat as a new column (e.g. group mean, group rank, normalized values within a group). Use `apply()` when you want a summary.

`groupby(observed=True)` — Categorical columns

When grouping by a **Categorical** column (e.g. one created by `pd.cut()`), pandas by default includes all possible categories — even ones with no data (`observed=False`).

`observed=True` means only show groups that actually appear in the data:

```
df.groupby('size', observed=False)['price'].mean()
# S     10.0
# M     15.0
# L      NaN   ← no data, shown anyway
# XL     NaN   ← no data, shown anyway

df.groupby('size', observed=True)['price'].mean()
# S     10.0
# M     15.0   ← only real groups
```

Always use `observed=True` when grouping by a column created with `pd.cut()` or `pd.qcut()` to avoid phantom empty groups and silence the FutureWarning.

Mapping a grouped result back to a DataFrame

After a `groupby().apply()`, the result is a Series indexed by the group key. Use `.map()` to add it back as a column:

```
gpa = merged.groupby('student_id').apply(
    lambda g: np.dot(g['score'], g['credits']) / g['credits'].sum()
)

students['gpa'] = students['student_id'].map(gpa)
```

`.map(series)` looks up each value in `student_id` against the Series index and fills in the corresponding value.

When to use `.to_numpy()`

Usually you don't need it — NumPy functions and pandas operations accept Series directly.

Only convert when:

- A library requires a plain array
- Using `np.dot()` or matrix operations where the index causes issues
- Something breaks and you suspect the index is the cause
- **Positional indexing with an array of indices** (e.g. after `np.argsort()`)

Label vs positional indexing:

```
aa = np.argsort(sales['revenue'])

sales['rep'][aa]            # label-based — works only if index is
0,1,2,3...
sales['rep'].to_numpy()[aa] # positional — always reliable
```

If the DataFrame has been filtered and the index is no longer `0, 1, 2...`, the first version gives wrong results or a KeyError. Always use `.to_numpy()` when indexing with an array of positions.

Concrete example — why it breaks:

```
df = pd.DataFrame({
    'name':   ['Alice', 'Bob', 'Carol', 'Dave'],
    'salary': [3000, 1000, 4000, 2000],
}, index=[10, 11, 12, 13])  # non-default index

aa = np.argsort(df['salary'])  # → [1, 3, 0, 2]  (positions)

df['name'][aa]            # looks for labels 1, 3, 0, 2 → KeyError!
df['name'].to_numpy()[aa] # looks for positions 1, 3, 0, 2 → correct
```

MultiIndex — set, sort, and slice

Creating a MultiIndex:

```
df = df.set_index(['region', 'quarter']).sort_index()
# region and quarter become index levels, no longer columns
```

Retrieving rows:

```
df.loc['West']                # all rows where first level == 'West'
df.loc[('East', 'Q3')]        # single row for specific (region, quarter)
combo
```

Slicing by second level — use `pd.IndexSlice` or `slice(None)` for "all" on a level:

```
idx = pd.IndexSlice
df.loc[idx[:, 'Q3'], :]       # all Q3 rows across every region (readable)
df.loc[(slice(None), 'Q3'), :] # same thing, without pd.IndexSlice
```

Mean per group using NumPy:

```
for region in df.index.get_level_values('region').unique():
    print(region, np.mean(df.loc[region, 'revenue']))
```

Re-grouping a MultiIndex result with `groupby(level=)`

After a multi-column `groupby`, the result has a **MultiIndex**. You can group it again using `level=` to operate within each top-level group.

Use case — find the best category per region:

```
# Step 1: sum revenue by region+category → MultiIndex Series
totals = ecommerce.groupby(['region', 'category'])['revenue'].sum()

# Step 2: re-group by level=0 (region), then idxmax within each region
totals.groupby(level=0).idxmax()
# North    ('North', 'Electronics')
# South    ('South', 'Home')
# ...
```

- `level=0` → first index level (region)
- `level=1` → second index level (category)

Compare to just `.idxmax()` with no re-groupby — that returns a single global max tuple, not one per region.

`groupby().agg()` with all numeric columns

`groupby().agg()` fails on non-numeric columns (e.g. strings). Use `select_dtypes` to limit to numeric columns first:

```
numeric_cols = df.select_dtypes(include='number').columns
df.groupby('species')[numeric_cols].agg(['mean', 'std'])
```

What doesn't work:

```
df.groupby('species').agg(['mean', 'std'])                      #
TypeError on object columns
df.groupby('species').agg(['mean', 'std'], numeric_only=True)    #
numeric_only not a valid agg kwarg
```

Avoiding SettingWithCopyWarning — `.copy()`

When you slice a DataFrame (e.g. with `.xs()`, `.loc[]`, boolean filter), the result may be a **view**, not a new DataFrame. Assigning to it raises a `SettingWithCopyWarning`:

```
q3 = sales_q.xs('Q3', level='quarter')
q3['total'] = q3['revenue'] * q3['units']  # SettingWithCopyWarning
```

Fix: call `.copy()` to make it an independent DataFrame:

```
q3 = sales_q.xs('Q3', level='quarter').copy()
q3['total'] = q3['revenue'] * q3['units']  # no warning
```

`.stack()` returns a Series, not a DataFrame

`.stack()` folds columns into rows — the result is a **Series** with a MultiIndex, not a DataFrame:

```
energy            # DataFrame — buildings as index, months as columns
energy_long = energy.stack()   # Series — index is (building, month),
values are numbers
```

Because it's a Series (one column of values), you can't add new columns to it directly. Use `.to_frame()` first:

```
energy_long = energy.stack().to_frame(name='kwh')   # now a DataFrame
energy_long['log_kwh'] = np.log(energy_long['kwh']) # can add columns now
```

`groupby('name')` vs `groupby(level='name')` — columns vs index levels

When you call `groupby('city')`, pandas looks for a **column** named `'city'` first. If it doesn't find one, it falls back to checking index level names.

This means `groupby('city')` can accidentally work after `set_index(['city', 'month'])` — because there's no longer a column called `'city'`, so pandas falls back to the index level.

Use `groupby(level='city')` to be explicit and unambiguous:

```
# After set_index(['city', 'month']):
weather.groupby('city')['temp'].max()         # works by coincidence —
city is no longer a column
weather.groupby(level='city')['temp'].max()   # explicit — always means
the index level
```

The difference matters if a column and an index level share the same name — pandas would group by the column, not the level.

`sort_values()` vs `sort_index()`

- **`sort_values()`** — sorts by the **data** in a column
- **`sort_index()`** — sorts by the **index labels**

```
s = pd.Series([30, 10, 20], index=['b', 'c', 'a'])

s.sort_values()
# c    10
# a    20
# b    30

s.sort_index()
# a    20
# b    30
# c    10
```

Common use case: after a `groupby`, the index is the group keys — `sort_index()` alphabetizes the groups, `sort_values()` ranks them by their aggregated value.

DateTime operations with `.dt`

Convert a string column to datetime, then use the `.dt` accessor to extract parts:

```
df['hire_date'] = pd.to_datetime(df['hire_date'])

df['year']  = df['hire_date'].dt.year
df['month'] = df['hire_date'].dt.month
```

Calculating days since a date:

```
df['tenure_days'] = (pd.Timestamp('today') - df['hire_date']).dt.days
```

Subtracting two datetime columns gives a `Timedelta` — `.dt.days` converts it to an integer.

Summarise with NumPy:

```
np.mean(df['tenure_days'])
np.median(df['tenure_days'])
```

This also happens after `.dropna()`, `.drop_duplicates()`, or any filter — the index keeps the original row numbers, so it's no longer `0, 1, 2...`:

```
df = df[df['salary'] > 1500]
# index is now [10, 12, 13] — df['name'][aa] will break
```

Assigning a Series with non-matching index to a DataFrame column:

```
# w has index ['Alice', 'Bob', 'Carol'] (from groupby key)
# employees has index [0, 1, 2]
employees['score'] = w            # index mismatch → NaN
employees['score'] = w.to_numpy() # assigns by position → correct
```

This happens whenever the Series index doesn't match the DataFrame index — common after groupby.

FutureWarning — what it means

pandas releases new versions over time and sometimes changes how things behave. A `FutureWarning` means:

> "This currently works, but we're going to change it in a future version. Update your code now."

If you upgrade pandas without fixing the warning, your code will break.

Common example — positional vs label indexing:

```
g['month'][np.argmax(g['revenue'])]  # FutureWarning — ambiguous: position
or label?
```

`np.argmax` returns an integer position. Passing it to `[]` is ambiguous — pandas currently treats it as a position, but will treat it as a label in future.

Fix by being explicit:

```
g['month'].iloc[np.argmax(g['revenue'])]   # clearly positional
g['month'].to_numpy()[np.argmax(g['revenue'])]  # outside pandas, no
ambiguity
```